



Programación I

14/07/2026

Examen Final

Docente: Dondero Nicanor

Consideraciones Iniciales

- Agregar al principio del proyecto un comentario con su apellido y nombre.
- Sólo se considera válida una función completamente comentada si es la versión iterativa de una función recursiva que **no** esté completamente comentada. De lo contrario, el código comentado no se corrige.
- Identifique con comentarios las funciones que corresponden a cada inciso.
- La falta de prolijidad restará puntaje. Utilice el formateador con frecuencia.
- Se les recuerda a los estudiantes que tendremos tolerancia cero al uso no autorizado de herramientas de IA así como al acceso a material ajeno al examen. Cualquier situación irregular puede invalidar el examen.

Narrativa

Se nos pide desarrollar una aplicación de consola en C que gestione un festival de música. Para ello se nos provee un modelo de datos y algunas funcionalidades que debemos implementar.

```
typedef struct stPresentacion
{
    int id;
    char nombreArtista[32];
    char genero[32];
    char escenario[32];
    int duracion;
    int esShowPrincipal;
} Presentacion;
```

Se detallan algunas validaciones importantes respecto del modelo de datos provisto:

- Las cadenas de texto, el nombre del artista, el género y el escenario no deberían quedar vacías, esto no es obligatorio pero es valorado.
- La duración debe estar comprendida entre 30 y 120 minutos.



- Un artista determinado no puede tener más de una presentación en cada escenario.
- Un artista determinado no puede tener más de un show principal, esto también es opcional pero valorado.
- El identificador único (id) es único por definición, por lo que no debe ser responsabilidad del usuario cargarlo manualmente.

A los efectos de no perder tiempo con la portabilidad del sistema y problemas con la entrada estándar se nos provee una librería que abstrae la entrada de datos por teclado (`scanner.h`). Esto implica que **no debe** utilizar `scanf()`, `gets()` ni `fgets()` por cuenta propia sino las variantes proporcionadas por la librería: `scanInt()`, `scanChar()`, `scanFloat()` y `scanString()`. Dicho todo esto, **no es obligatorio** el uso de la librería.

También se provee la librería de pilas para hacer operaciones con esa estructura de datos.

Consignas y puntaje

Si se presenta un prototipo de una función, la función debe ser realizada de esa manera para obtener la totalidad del puntaje. En el caso de las funciones recursivas, si se implementan de manera iterativa recibirán una fracción del puntaje pero no se considerarán incorrectas. Para la persistencia de datos se utilizará un archivo binario, para lo cual es recomendable definir una constante global con el nombre del archivo. Es responsabilidad del estudiante darse cuenta de qué funciones deberían imprimir mensajes (o solicitar datos por teclado), es decir lógica de presentación y qué funciones no deberían hacerlo, es decir lógica de negocio. El alto acoplamiento de una función reduce el puntaje, debe modularizar correctamente.

Valor	Inciso
10	1. A los efectos de evaluar la algoritmia clásica de obtener un valor mínimo o máximo, debe implementar una función que retorne el siguiente ID válido para que el usuario no tenga que cargarlo manualmente. Se conoce que un valor menor o igual a 0 (cero) es inválido, por lo que el primer id válido sería el 1 (uno). No hay garantía de que el archivo de presentaciones esté ordenado por id por lo que el algoritmo debe retornar el siguiente al máximo id y no el siguiente al id del último registro.
10	2. Implemente la validación para que un artista no pueda tener más de una presentación por escenario. <code>int tieneOtroShowEnMismoEscenario(char nombreArchivo[], char nombreArtista[], char escenario[]);</code>
10	3. Implemente la función que solicita al usuario los datos para crear una presentación. Debe retornar una presentación válida, por lo que llamará a todas las funciones de validación que haya codificado. Como esta función realiza acciones de validación con datos del disco, es recomendable que reciba el nombre del archivo para pasarlo a las funciones que necesitan esa información.



10	4. Implemente una función que cargue un arreglo estático de presentaciones de capacidad limitada pero parametrizada, es decir que se carguen tantas presentaciones como el usuario quiera sin excederse de la dimensión dada.
10	5. Implemente una función que guarde un arreglo de presentaciones en un archivo binario. El programa va a tratar al arreglo estático anterior como un buffer y por tanto éste debe vaciarse hacia el archivo, por lo que debe modificar los válidos por referencia y devolverlos a 0 (cero) para permitir su reutilización. Esta función no debe iterar sobre el arreglo sino escribir todos los datos en el archivo con una única instrucción.
5	6. Implemente una función recursiva que muestre un arreglo de presentaciones, modularizar . Debe mostrar los datos en orden (del primero al último) y llevar el siguiente prototipo: <code>void mostrarPresentaciones(Presentacion presentacion[], int validos);</code>
5	7. Implemente una función recursiva que muestre todas las presentaciones que sean show principal del archivo binario.
15	8. Implemente las funciones que crea necesarias para traer a memoria un arreglo dinámico del tamaño justo con todas las presentaciones de un escenario determinado. Piense en la modularización y en los parámetros que deben recibir las funciones. La memoria dinámica debe ser solicitada dentro de una de estas funciones y no dentro del main(). La función principal debe tener el siguiente prototipo: <code>Presentacion* cargarPresentacionesDeUnEscenario(char nombreArchivo[], char escenario[], int* validos);</code>
15	9. Implemente las funciones que crea necesarias para reportar la duración promedio de cada género. Para ello se utilizará un arreglo de palabras y pilas. De ninguna manera será un algoritmo eficiente, de hecho es la estructura de datos equivocada para realizar esta tarea pero es requisito resolver esta problemática de la siguiente manera: a. La función principal (la que se llama desde el main para realizar todo este procedimiento) debe contener una matriz de char local que pueda contener al menos 10 géneros y sabemos que los géneros son cadenas de hasta 32 caracteres. b. Se debe abrir el archivo en busca de cada género para copiarlo a dicha matriz, para ello se debe verificar que no exista ese género en el arreglo de palabras para no tener géneros duplicados. c. Una vez se tienen todos los géneros del archivo se debe volver a recorrer el archivo con el primer género encontrado y guardar las duraciones de las presentaciones que corresponden a ese género en una Pila. d. Finalizado este recorrido se debe vaciar la Pila hacia un acumulador al mismo tiempo que se cuentan los elementos de la Pila. e. Con toda esa información se calcula la duración promedio y se imprime el resultado. f. Una vez llegado a este paso se sigue con el siguiente género encontrado, lo que significa que se va a recorrer el archivo tantas veces como géneros haya (más la primera vez que fue necesaria para encontrarlos).



10	10. Implemente la función main() que llame directa o indirectamente a todas las funciones codificadas. El flujo de la aplicación (que es un entorno de pruebas secuencial, no un programa con menú) debe garantizar que: Siempre al iniciar la aplicación se proceda a cargar al menos una presentación. - Al finalizar la carga se muestren todos los datos cargados y se pregunte si se desean guardar los datos del arreglo al archivo. Si se guardan los datos se preguntará si se desean cargar más datos. - Se le pregunte al usuario por el nombre de un escenario y a continuación se traigan a memoria todas las presentaciones de ese escenario y se muestren si las hay o se informe que no hay en caso contrario. - Se muestren todas las presentaciones principales. - Se reporten las duraciones promedio de cada género.
----	---